A Cognitive Load Theory Perspective on AI-Assisted Design Recipe Environments

Keerthana Kompella, University of Washington, Seattle
Dr. Nadia Najjar, College of Computing and Informatics, UNC Charlotte

Introduction

Problem

- Learning to program is difficult
- Hard to get novices to follow structured frameworks
- Need to support computational thinking

Proposed Solution

- AI tool to help novices follow a structured framework

Research Question: How can Cognitive Load Theory inform the interface design of AI coding tutors to better support the iterative steps of the Design Recipe?

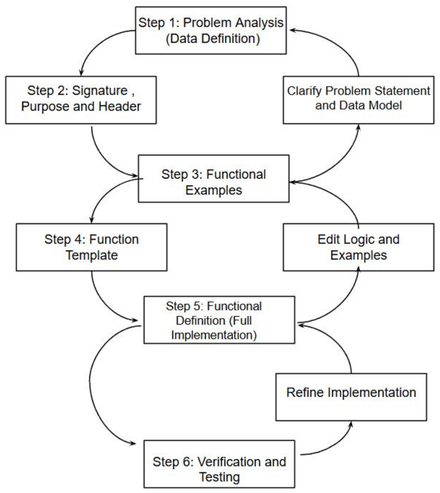
Background and Literature Review

Cognitive Load Theory (CLT)

- Theory that human working memory is limited (Sweller 1988)
- Digital Environments
  - Increase cognitive load through bad interface design and unstructured feedback

The Design Recipe (DR)

- Framework that prioritizes design and problem solving over syntax (Felleisen et al. 2004)
- Challenges:
  - DR treated as a checklist
  - Novices struggle with terminology



Generative AI

- Positives:
  - Shifts novice's burden of memorizing syntax to problem decomposition
- Negatives:
  - Use of AI assistance fails to translate into fundamental problem solving skills - "hollowing of competence" (Zhu & Liu, 2026)
  - AI often does heavy lifting for students
  - Pushes students into the role of "code reviewer"

Proposed Framework

1. **Spatial and Temporal Alignment → Integrating DR steps**
   - **Definition:** Related information should align physically and chronologically
   - **Implementation:** In-context non modal feedback
2. **Artifact Interdependence → Breaking checklist mentality**
   - **Definition:**The interface should visually map the connection between steps
   - **Implementation:** Explicit cross step dependency linking
3. **Visual Cues → Encouraging nonlinear processes**
   - **Definition:** The system should make suggested problem flow visible
   - **Implementation**: Visual Iterativeness
4. **Mental Model Alignment → Scaffolding terminology**
   - **Definition:** The system should adapt to the student's mental model
   - **Implementation:** Adaptive feedback
5. **Pedagogical Transparency → Encouraging problem decomposition**
   - **Definition:** Tool should have clear and transparent processes
   - **Implementation:** Human-AI partnership

Evaluation of Existing AI Tools Employing Design Recipe

FEEDBOT (Zhu et al., 2025)

- LLM evaluates design steps and returns short hints
- Each step of the DR evaluated in isolation

Ladder (Kumamoto et al., 2025)

- Interactive natural language dialogue
- Writes final code for student

SmartCode Tutor (Lamin et al., 2026)

- Guides students through a plan-implement-test-debug loop
- Relies on student managed conversational prompts

Heuristic	FEEDBOT	Ladder	SmartCodeTutor
Spatial and Temporal Alignment	○	⊘	⊘
Artifact Interdependence	⊘	✓	○
Visual Cues	⊘	⊘	○
Mental Model Alignment	⊘	✓	✓
Pedagogical transparency	⊘	⊘	⊘

Table 1: Evaluation of the three existing AI design recipe environments using the proposed framework

⊘ = No ✓ = Yes ○ = Partial

Conclusions and Future Work

Conclusions

- Interface layer as a bottleneck
  - Well structured interfaces reduce cognitive load
- Preventing competency hollowing
  - AI scaffolding improves role of AI in learning
- Redefining novices' role
  - Introduction of AI changes novice role
  - Computational thinking highly important

Benefits of Paper to CS Education

- Actionable HCI Guidelines
  - Concrete theory driven rubric
- Contribute to DR literature
- Analyzes the intersection between DR, AI and CLT

Future Work

- Further Literature Review
- Heuristic study to evaluate proposed framing
- Development of tool that fits these design heuristics
- User Testing

References

- Felleisen, M., Findler, R. B., Flatt, M., & Krishnamurthi, S. (2004). The TeachScheme! Project: Computing and education. *Journal of Functional Programming*, 14(4), 365-398
- Kumamoto, K., Cong, Y., & Masuhara, H. (2025). An interactive learning environment for program design. In Proceedings of the 2025 ACM SIGPLAN International Symposium on SPLASH-E (SPLASH-E '25) (pp. 66–74). Association for Computing Machinery. https://doi.org/10.1145/3758317.3759682
- Lamin, N. Z., Wan Jusoh, W. N. A., Wan Muhamad Tahir, W. A., Miserom, S. F., & Mohd Zin, A. (2026). Design and pilot evaluation of SmartCode Tutor (SCT): A mixed-methods framework for AI-assisted introductory programming in higher education. Proceedings of the International Conference on Cross-Disciplinary Academic Research 2025 - Track 1 Advances in Computing, Electronics, Engineering, and Mathematics (ICAR-T1 2025), 81–90. Atlantis Press. https://doi.org/10.2991/978-94-6239-636-4_8
- Sweller, J. (1988). Cognitive load during problem solving: Effects on learning. Cognitive Science, 12(2), 257–285. https://doi.org/10.1207/s15516709cog1202_4
- Zhu, E., Teja, S., Coombes, C., & Patterson, D. (2025). FEEDBOT: Formative design feedback on programming assignments. In *Proceedings of the 30th ACM Conference on Innovation and Technology in Computer Science Education V. 1*. Association for Computing Machinery. https://doi.org/10.1145/3724363.3729063